

Week 1

Concept

Q: What is the era of multicore machines, and why did software have to change?
A: Clock-speed scaling hit power, heat, and diminishing-return limits, so hardware vendors added cores instead. Programs no longer speed up automatically; developers must expose parallelism, manage synchronization, balance work, and design for scalability.

Q: How do shared-memory and distributed-memory systems differ, and why does this matter?
A: Shared memory uses threads that see one address space, so communication happens through shared variables and synchronization. Distributed memory uses separate processes with private memory, so communication is explicit, usually with MPI. The machine model determines decomposition, data placement, and communication cost.

Q: How does Gustafson-Barsis differ from Amdahl's Law?
A: Amdahl assumes fixed-size problems and says the serial fraction caps speedup: $S(p) = 1 / ((1 - \alpha) + \alpha/p)$. Gustafson assumes scaled workloads and gives $S(p) = (1 - \alpha) + \alpha p$, so larger machines can solve larger problems efficiently. Amdahl is pessimistic for fixed work; Gustafson is more optimistic for scaled work.

Model	Best fit
Amdahl	Fixed-size workload; serial part limits speedup
Gustafson	Scaled workload; more processors solve more work

Analysis

Q: Why might a single-core application gain little on a multicore processor?
A: Common causes are limited parallelism, a large serial fraction, synchronization overhead, load imbalance, data dependencies, thread management overhead, memory bandwidth limits, and cache contention. Good next step: profile first, then refactor the sequential or contended hot spots.

Factor	Impact
Serial work	Caps possible speedup
Locks/barriers	Threads wait instead of compute
Imbalance	Some cores go idle early
Bandwidth/cache	Contention lowers scaling

Q: Can a program with a large serial fraction still scale to thousands of cores?
A: Under Amdahl, usually no: a large serial fraction causes speedup saturation. Under Gustafson, possibly yes if the workload scales with processor count so the parallel part grows and the serial part becomes a smaller fraction of total time. The claim is weak for fixed-size problems and stronger for scaled workloads.

Programming

Q: A sequential program takes 120 s and a parallel version takes 30 s on 8 cores. What are speedup and efficiency?
A: Use $S = T_{seq} / T_p$ and $E = S / p$. Here $S = 120 / 30 = 4$ and $E = 4 / 8 = 0.5 = 50\%$. Interpretation: useful speedup, but sublinear, so overhead or serial work remains.

Q: Given speedups 1.0, 1.8, 3.2, 5.0 on 1, 2, 4, 8 cores, what is the scalability story?
A: Efficiencies are 1.00, 0.90, 0.80, and 0.625. Scaling is sublinear and diminishing returns are clear by 4 to 8 cores.

Cores	S	E
1	1.0	100%
2	1.8	90%
4	3.2	80%
8	5.0	62.5%

Q: What pseudocode should you write for a speedup/efficiency question?
A: Read `Tseq`, `Tpar`, and `cores`; compute `speedup = Tseq / Tpar`; compute `efficiency = speedup / cores`; print all inputs plus speedup and efficiency.

Week 2

Concept

Q: What is a thread, and how is it different from a process?
A: A thread is a lightweight execution unit inside a process. Threads share memory and resources, so communication is cheap but synchronization is required. Processes are isolated and use heavier inter-process communication.

Q: What are atomic data types or atomic operations, and why do they help?
A: Atomic operations appear to happen as one indivisible step. They prevent unsafe interleavings for simple shared updates such as counters or flags, so `counter++` can be made safe when performed atomically. For larger multi-step critical sections, use a mutex instead.

Q: Semaphore-based producer-consumer vs monitor-based producer-consumer?
A: Semaphore solutions use explicit counters such as empty/full slots and usually a mutex around the buffer. Monitor-style solutions wrap the shared buffer and synchronization inside one class with methods like `produce()` and `consume()`, which improves encapsulation and reduces ordering mistakes.

Analysis

Q: Eight threads update one global counter. What can go wrong?
A: Race conditions, lost updates, non-atomic `counter++`, and visibility problems. The final result becomes timing-dependent and may change across runs. Fix with `std::atomic<int>` for simple increments or a mutex for larger critical sections.

Problem	Impact
Race condition	Final result depends on timing
Lost update	Some increments disappear
Non-atomic ++	Read/add/write can interleave
Visibility issue	Threads may see stale data

Q: Why is thread speedup usually not linear as threads increase?
A: Amdahl's serial fraction stays, while lock contention, memory bandwidth limits, load imbalance, and scheduling overhead grow. If time drops only slightly from 8 to 16 threads, the program is near saturation.

Programming

Q: How do you write a safe shared-counter program?
A: Create multiple threads, let each increment a shared counter many times, and protect the increment with `std::lock_guard<std::mutex>` or use `std::atomic<int>`. The unsafe version can print the wrong final count; the safe version should match the expected count.

Q: How do you implement producer-consumer using a monitor-like construct?
A: Wrap `queue`, `mutex`, and `condition_variables` inside a `BoundedBuffer` class. In `produce()`, wait until space exists, push, then notify consumers. In `consume()`, wait until data exists, pop, then notify producers.

Week 3

Concept

Q: What is divide-and-conquer, and why is it effective for recursive problems?
A: Break the problem into smaller subproblems, solve them independently, often in parallel, and combine the results. Recursive problems naturally fit this because subproblems are often independent at each level.

Q: What is the master-worker pattern?
A: A master creates tasks, distributes them to workers, tracks progress, and collects results. Workers repeatedly receive work, compute, and return outputs. It fits job queues, image processing, data analysis, and other workloads with many similar independent tasks.

Q: Why must the chosen parallel pattern match the problem structure?
A: A good pattern matches decomposition, dependencies, and data layout. The wrong pattern creates unnecessary communication, poor load balance, excess synchronization, idle processors, and weak scalability.

Analysis

Q: Why is choosing a parallel program harder than designing a sequential one?
A: Parallel design must preserve correctness while exposing useful concurrency. The main issues are task decomposition, dependencies, synchronization, communication, load balance, memory consistency, scalability, and task granularity.

Q: How do you apply PCAM to image processing?
A: Partition the image into rows or tiles, communicate halo or boundary pixels when needed, agglomerate tiny pixel tasks into larger tiles to reduce overhead, and map tiles evenly to processors or threads.

PCAM	Image example
Partition	Rows/tiles/regions
Communicate	Boundary or halo pixels
Agglomerate	Merge tiny tasks into tiles
Map	Spread tiles evenly

Programming

Q: How do you implement a three-stage pipeline?
A: Use one thread each for read, process, and store, with thread-safe queues between stages. The reader pushes data into queue 1, the processor waits on queue 1, transforms items, and pushes to queue 2, and the writer waits on queue 2 and outputs results.

Q: How do you implement a master-worker application?
A: Use a shared task queue, one or more worker threads, and synchronization around queue access. The master pushes tasks and signals availability; workers wait for tasks, pop one safely, compute, and store the results.

Week 4

Concept

Q: What is MPI and why is it common on distributed-memory systems?
A: MPI is a standard message-passing interface for parallel processes. Distributed-memory systems have no shared address space, so data must be exchanged explicitly; MPI provides scalable point-to-point and collective communication for that model.

Q: What do `MPI_Comm_rank()` and `MPI_Comm_size()` do?
A: `MPI_Comm_rank()` gives the current process ID in the communicator, and `MPI_Comm_size()` gives the total number of participating processes. They let processes choose roles, split data, and target communication correctly.

Q: What is buffered communication in MPI?
A: Buffered send copies the outgoing message into a user-attached buffer, so the sender can continue after the copy completes. It differs from standard blocking `MPI_Send`, where MPI may block the sender before the send buffer is safe to reuse.

Analysis

Q: How can one MPI executable behave differently on different processes in SPMD?
A: All ranks run the same program image, but each process learns its rank and total size, branches on rank, and works on different local data. The same executable therefore yields root roles, worker roles, and different communication paths.

Mechanism	Role in SPMD
Rank	Gives each process an identity
Size	Sets total participants
Branch on rank	Root and worker behavior
Data split	Same code, different local work

Programming

Q: What should a minimal MPI greeting program do?
A: Call `MPI_Init`, get `rank` and `size` with `MPI_Comm_rank` and `MPI_Comm_size`, print a greeting like "Hello from rank ...", then call `MPI_Finalize`.

Q: How do you modify a blocking `MPI_Send` program to use buffered communication?
A: Attach a user buffer with `MPI_Buffer_attach`, replace `MPI_Send` with `MPI_Bsend`, then call `MPI_Buffer_detach` before exit. The receiver can still use `MPI_Recv`.

Week 5

Concept

Q: What is collective communication, and how is it different from point-to-point communication?
A: Collectives involve a whole communicator in one coordinated operation such as broadcast, scatter, gather, reduce, or all-to-all. Point-to-point communication connects exactly one sender and one receiver and requires manual coordination.
Q: What is `MPI_Gather()`, and why is it common?
A: It collects local chunks from all ranks and stores the combined result on one root rank in rank order. It is common in scatter-compute-gather patterns where each process works independently on a piece of data and the root needs the full output.
Q: Why is all-to-all communication more expensive than scatter or gather?
A: Scatter is one-to-many, gather is many-to-one, but all-to-all is many-to-many. That means more total traffic, more coordination, and more chance of contention, so `MPI_Alltoall` is usually the most communication-intensive choice.
Analysis
Q: A large array on rank 0 must be divided equally among all processes. Which operation should you choose and why?
A: Use `MPI_Scatter`. It is designed to split one root buffer into equal-sized chunks and distribute one chunk to each rank. It is clearer and usually better optimized than manual send/receive code.

Programming

Q: How do you write an `MPI_Scatter()` distribution program?
A: Create a full array on rank 0, allocate a small receive buffer on each rank, call `MPI_Scatter`, then print the received values locally on each process.
Q: How do you compute a global average with MPI?
A: Pattern: Scatter -> local sum -> Reduce. Scatter equal chunks, compute `localSum`, combine with `MPI_Reduce(..., MPI_SUM, 0, ...)`, and on rank 0 compute `average = (double)globalSum / N;`

Need	MPI call
Same data everywhere	<code>MPI_Bcast</code>
One chunk per rank	<code>MPI_Scatter</code>
Collect chunks to root	<code>MPI_Gather</code>
Combine values	<code>MPI_Reduce</code>
Everyone exchanges	<code>MPI_Alltoall</code>

Week 6

Concept

Q: What is CUDA, and why is it important?
A: CUDA is NVIDIA's GPU computing platform and programming model. It lets the CPU launch kernels on the GPU, which can run thousands of lightweight threads on large data-parallel workloads such as simulations, image processing, and machine learning.
Q: What is a warp, and why is it significant?
A: A warp is a group of 32 CUDA threads scheduled together. Warps are the basic GPU execution unit, help hide latency, and work best when threads follow the same control flow. Divergence forces different branch paths to execute separately and lowers throughput.
Q: What is the role of Thrust algorithms? Give examples.
A: Thrust gives STL-like high-level GPU algorithms, so common parallel operations need less low-level CUDA code. `thrust::transform` applies an operation elementwise, and `thrust::reduce` combines many values into one, such as summing a vector or aggregating results.

Analysis

Q: Is a workload with millions of independent elements a good CUDA candidate?
A: Usually yes. It has strong data parallelism and high task independence, which fit CUDA well. The main caution is transfer overhead: if host-device copies dominate the runtime, gains may shrink or disappear.

Factor	CUDA implication
Data parallelism	Strong fit
Task independence	Low synchronization cost
SIMT-friendly flow	Better warp efficiency
High transfer cost	Can erase gains
Enough work	Helps amortize overhead

Programming

Q: What does a minimal GPU hello-world program look like?
A: Write a `__global__` kernel that prints a message, launch it with something like `helloKernel<<1,4>>>()`, and call `cudaDeviceSynchronize()` so the host waits for GPU output.
Q: How do custom CUDA vector addition and Thrust vector addition compare?
A: A custom kernel gives low-level control over indexing, launch, and memory management using lines like `int i = blockIdx.x * blockDim.x + threadIdx.x;` and if (`i < N`) `C[i] = A[i] + B[i];`. Thrust is shorter for common operations, for example `thrust::transform(A.begin(), B.begin(), C.begin(), thrust::plus<int>());`.

Approach	Control	Best use
Kernel	High	Custom tuning
Thrust	Lower	Common data-parallel ops

Q: What is the basic CUDA workflow?
A: Allocate device memory with `cudaMalloc`, copy host to device with `cudaMemcpyHostToDevice`, launch the kernel, copy results back with `cudaMemcpyDeviceToHost`, then free memory. A common indexing line is `int i = blockIdx.x * blockDim.x + threadIdx.x;`

Code Samples

Week 1: Speedup Pseudocode

```
INPUT Tseq
INPUT Tpar
```

```
INPUT cores
speedup = Tseq / Tpar
efficiency = speedup / cores
PRINT speedup, efficiency
```

Week 2: Mutex Counter

```
void increment(){
    for(int i=0; i<50000; i++){
        std::lock_guard<std::mutex> lock(mtx);
        counter++; // shared variable
    }
}
```

Week 2: Semaphore Producer

```
slotsAvailable.acquire(); // wait for empty slot
{
    std::lock_guard<std::mutex> lock(mutobj);
    buffer[in] = item;
    in = (in + 1) % BUFFSIZE;
}
resAvailable.release();
```

Week 3: Pipeline Processor

```
std::unique_lock<std::mutex> lock1(mtx1);
cv1.wait(lock1, []{ return !q1.empty() || doneProducing; });
int x = q1.front();
q1.pop();
lock1.unlock();
int result = x*x;
```

Week 3: Master-Worker

```
std::lock_guard<std::mutex> lock(mtx);
if(tasks.empty())
    return;
task = tasks.front();
tasks.pop();
std::cout << "Worker " << id << " processing task " << task;
```

Week 3: Fork-Join

```
std::thread t1([&leftSum, start, mid](){
    leftSum = parallelSum(start, mid);
});
std::thread t2([&rightSum, mid, end](){
    rightSum = parallelSum(mid, end);
});
t1.join();
t2.join();
return leftSum + rightSum;
```

Week 4: MPI Hello Skeleton

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
std::cout << "Hello from rank " << rank << " of " << size;
MPI_Finalize();
```

Week 4: MPI Send/Recv

```
if (rank == 0) {
    int value = 42;
    MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    int value = 0;
    MPI_Status status;
    MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
}
```

Week 4: Buffered Send Steps

```
int bufferSize = MPI_BSEND_OVERHEAD + N * sizeof(int);
char* buffer = new char[bufferSize];
MPI_Buffer_attach(buffer, bufferSize);
MPI_Bsend(data, N, MPI_INT, 1, 0, MPI_COMM_WORLD);
MPI_Buffer_detach(&detached, &detachedSize);
```

Week 5: Scatter and Gather

```
MPI_Scatter(sendbuf, 2, MPI_INT, recvbuf, 2, MPI_INT, 0, MPI_COMM_WORLD);
recvbuf[0] += 2;
recvbuf[1] += 2;
MPI_Gather(recvbuf, 2, MPI_INT, gatherbuf, 2, MPI_INT, 0, MPI_COMM_WORLD);
```

Week 5: Reduce for Average

```
int localSum = 0;
for (int i = 0; i < valuesPerProcess; i++)
    localSum += recvbuf[i];
MPI_Reduce(&localSum, &globalSum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

Week 6: GPU Hello

```
__global__ void helloKernel() {
    printf("Hello from GPU thread %d\n", threadIdx.x);
}
helloKernel<<1, 4>>>();
cudaDeviceSynchronize();
```

Week 6: CUDA Kernel

```
__global__
void add(int *A, int *B, int* C, int N){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < N)
        C[i] = A[i] + B[i];
}
add<<1,256>>>(d_A, d_B,d_C,N);
```

Week 6: Thrust

```
thrust::device_vector<int> v(5);
int sum = thrust::reduce(v.begin(),v.end());
thrust::transform(A.begin(), B.begin(), C.begin(), thrust::plus<int>());
```